

Representasi Tahapan Kompilasi Menggunakan Deklarasi Fungsi

Nama : Ahmad Reza
NIM : 231011403385
Mata Kuliah : Teknik Kompilasi

1. Deskripsi Program

Program ini dibuat untuk mensimulasikan tahapan proses kompilasi terhadap konstruksi deklarasi fungsi menggunakan bahasa pemrograman *Python*. Program tidak menghasilkan bahasa mesin, tetapi hanya memperlihatkan bagaimana sebuah *compiler* memproses *source code* mulai dari analisis leksikal hingga menghasilkan *Three Address Code* (TAC).

Konstruksi yang digunakan sebagai input adalah Deklarasi Fungsi.

```
int tambah(int a, int b){  
    return a + b;  
}
```

Program dibagi menjadi empat tahapan utama, yaitu:

- Analisis Leksikal (*Lexical Analysis*)
- Analisis Sintaksis (*Syntax Analysis*)
- Analisis Semantik (*Semantic Analysis*)
- Generasi *Three Address Code* (TAC)

2. Pola Tata Bahasa (Grammar)

Grammar ditulis menggunakan bentuk sederhana (BNF) sebagai berikut.

```
<function> ::= <type> <identifier> "(" <parameter_list> ")" "{" <return_stmt>  
    "}"  
  
<parameter_list> ::= <parameter> | <parameter> "," <parameter_list>  
  
<parameter> ::= <type> <identifier>  
  
<return_stmt> ::= "return" <expression> ";"  
  
<expression> ::= <identifier> "+" <identifier>  
  
<type> ::= "int" | "float" | "char"
```

Grammar tersebut mendefinisikan bahwa sebuah fungsi terdiri atas tipe data, nama fungsi, daftar parameter, serta sebuah pernyataan return.

3. Implementasi Program

Program dibuat menggunakan bahasa *Python* dengan pendekatan pemrograman berorientasi objek (*Object-Oriented Programming/OOP*). Seluruh proses kompilasi disimpan di dalam kelas *FunctionCompiler*.

Program terdiri dari empat fungsi utama.

a. *Lexical Analysis*

Fungsi ini menggunakan *Regular Expression* (Regex) untuk memecah *source code* menjadi token-token yang lebih kecil.

Regex yang digunakan:

```
token_pattern = r'int|float|char|return|[A-Za-z_]\w*|\+|\(|\)|\{|\}|,|;|'
```

Regex tersebut mengenali:

Bentuk	Jenis Token
int	Keyword
float	Keyword
char	Keyword
return	Keyword
tambah	Identifier
a	Identifier
b	Identifier
+	Symbol
()	Symbol
{}	Symbol
,	Symbol
;	Symbol

Setelah Regex menemukan semua lexeme, program mengubahnya menjadi token menggunakan proses klasifikasi.

Contoh hasil tokenisasi

(*KEYWORD*, *int*)

(*IDENTIFIER*, *tambah*)

(*SYMBOL*, *()*)

(*KEYWORD*, *int*)

(*IDENTIFIER*, *a*)

...

Tahap ini bertujuan mengubah *source code* menjadi bentuk yang lebih mudah diproses pada tahap berikutnya.

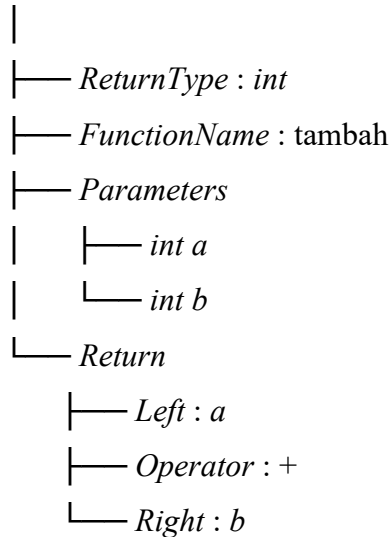
b. *Syntax Analysis*

Fungsi ini mengambil daftar token hasil analisis leksikal kemudian menyusunnya menjadi *Abstract Syntax Tree (AST)* sederhana.

AST disimpan menggunakan struktur *Dictionary Python*.

Contoh AST:

Function



AST menggambarkan struktur logis dari program tanpa memperhatikan karakter seperti tanda kurung atau titik koma.

c. *Semantic Analysis*

Tahapan ini melakukan beberapa pemeriksaan terhadap AST.

Pemeriksaan yang dilakukan meliputi:

- *Return type* harus valid.
- Nama parameter tidak boleh sama.
- Struktur fungsi harus sesuai.

Apabila seluruh pemeriksaan berhasil maka program menampilkan:

```
===== SEMANTIC ANALYSIS =====
Semantic Analysis Berhasil
```

d. *Three Address Code* (TAC)

Tahap ini menghasilkan representasi kode antara (*Intermediate Code*).

Output TAC yang dihasilkan adalah:

```
===== THREE ADDRESS CODE =====  
BeginFunc tambah  
t1 = a + b  
return t1  
EndFunc
```

Keterangan:

- *BeginFunc* menunjukkan awal fungsi.
- t1 merupakan variabel sementara (*temporary variable*).
- *return* t1 mengembalikan hasil operasi.
- *EndFunc* menunjukkan akhir fungsi.

Representasi ini lebih mudah diterjemahkan ke bahasa *assembly* dibandingkan *source code* asli.

4. Hasil Pengujian

Source Code

```
import re  
class FunctionCompiler:  
    def __init__(self, source_code):  
        self.source_code = source_code  
  
    # =====  
    # LEXICAL ANALYSIS  
    # =====  
    def lexical_analysis(self):  
        token_pattern = r'int|float|char|return|[A-Za-z_]\w*|\+|\(|\)|\{|\}|,|;'  
        raw_tokens = re.findall(token_pattern, self.source_code)  
        tokens = []  
        keywords = ["int", "float", "char", "return"]  
        for token in raw_tokens:  
            if token in keywords:  
                tokens.append(("KEYWORD", token))  
            elif token in ['(', ')', '{', '}', '!', '!', '!', '+']:  
                tokens.append(("SYMBOL", token))  
            else:  
                tokens.append(("IDENTIFIER", token))  
        return tokens
```

```

# =====
# SYNTAX ANALYSIS (AST)
# =====
def syntax_analysis(self):
    tokens = [t[1] for t in self.lexical_analysis()]
    ast = {
        "Function": {
            "ReturnType": tokens[0],
            "FunctionName": tokens[1],
            "Parameters": [
                {
                    "Type": tokens[3],
                    "Name": tokens[4]
                },
                {
                    "Type": tokens[6],
                    "Name": tokens[7]
                }
            ],
            "Return": {
                "Left": tokens[11],
                "Operator": tokens[12],
                "Right": tokens[13]
            }
        }
    }
    return ast

# =====
# SEMANTIC ANALYSIS
# =====
def semantic_analysis(self):
    ast = self.syntax_analysis()
    params = ast["Function"]["Parameters"]
    names = [p["Name"] for p in params]
    if len(names) != len(set(names)):
        return "Error: Nama parameter duplikat"
    if ast["Function"]["ReturnType"] not in ["int", "float", "char"]:
        return "Error: Return type tidak valid"
    return "Semantic Analysis Berhasil"

```

```

# =====
# THREE ADDRESS CODE
# =====
def generate_TAC(self):
    ast = self.syntax_analysis()
    left = ast["Function"]["Return"]["Left"]
    right = ast["Function"]["Return"]["Right"]
    tac = []
    tac.append(f"BeginFunc {ast['Function']['FunctionName']}")
    tac.append(f"t1 = {left} + {right}")
    tac.append("return t1")
    tac.append("EndFunc")
    return tac

# =====
# SOURCE CODE
# =====
source = """
int tambah(int a, int b){
    return a + b;
}
"""

compiler = FunctionCompiler(source)

print("===== SOURCE CODE =====")
print(source)

print("\n===== LEXICAL ANALYSIS =====")
tokens = compiler.lexical_analysis()
for token in tokens:
    print(token)

print("\n===== ABSTRACT SYNTAX TREE =====")
ast = compiler.syntax_analysis()
from pprint import pprint
pprint(ast)

print("\n===== SEMANTIC ANALYSIS =====")
print(compiler.semantic_analysis())

print("\n===== THREE ADDRESS CODE =====")
for line in compiler.generate_TAC():
    print(line)

```

Output

```
===== SOURCE CODE =====

int tambah(int a, int b){
    return a + b;
}

===== LEXICAL ANALYSIS =====
('KEYWORD', 'int')
('IDENTIFIER', 'tambah')
('SYMBOL', '(')
('KEYWORD', 'int')
('IDENTIFIER', 'a')
('SYMBOL', ',')
('KEYWORD', 'int')
('IDENTIFIER', 'b')
('SYMBOL', ')')
('SYMBOL', '{')
('KEYWORD', 'return')
('IDENTIFIER', 'a')
('SYMBOL', '+')
('IDENTIFIER', 'b')
('SYMBOL', ';')
('SYMBOL', '}')

===== ABSTRACT SYNTAX TREE =====
{'Function': {'FunctionName': 'tambah',
               'Parameters': [{'Name': 'a', 'Type': 'int'},
                              {'Name': 'b', 'Type': 'int'}],
               'Return': {'Left': 'a', 'Operator': '+', 'Right': 'b'},
               'ReturnType': 'int'}}

===== SEMANTIC ANALYSIS =====
Semantic Analysis Berhasil

===== THREE ADDRESS CODE =====
BeginFunc tambah
t1 = a + b
return t1
EndFunc

=== Code Execution Successful ===
```

5. Kesimpulan

Program yang dibuat berhasil mensimulasikan tahapan dasar proses kompilasi pada konstruksi deklarasi fungsi. Program mampu melakukan analisis leksikal untuk menghasilkan token, analisis sintaksis untuk membentuk *Abstract Syntax Tree* (AST), analisis semantik untuk memvalidasi struktur fungsi, serta *menghasilkan Three Address Code* (TAC) sebagai representasi kode antara. Dengan implementasi ini, konsep dasar proses kompilasi dapat dipahami secara lebih jelas melalui simulasi menggunakan bahasa *Python*.